

SocraticAI: Transforming LLMs into Guided CS Tutors Through Scaffolded Interaction

Karthik Sunil and Aalok Thakkar

Ashoka University
{karthik.sunil_ug25, thakkar}@ashoka.edu.in

Abstract. We present **SocraticAI**, a scaffolded AI tutoring system that integrates large language models (LLMs) into undergraduate Computer Science education through structured constraints rather than prohibition. The system enforces well-formulated questions, reflective engagement, and daily usage limits while providing Socratic dialogue scaffolds. Unlike traditional AI bans, our approach cultivates responsible and strategic AI interaction skills through technical guardrails including authentication, query validation, structured feedback, and RAG-based course grounding. Initial deployment demonstrates that students progress from vague help-seeking to sophisticated problem decomposition within 2–3 weeks, with over 75% producing substantive reflections and displaying emergent patterns of deliberate, strategic AI use.

1 Area and Context

The rapid adoption of LLMs has transformed programming workflows but also introduced profound pedagogical challenges. In undergraduate CS education, unrestricted AI access often leads to shallow dependency and solution copying rather than genuine conceptual understanding [1]. Institutional responses have largely polarized between strict prohibition and unrestricted freedom, neither of which effectively balance innovation with academic integrity. Prior research indicates that AI tutors can enhance learning when students remain actively engaged and reflective [8,7,12,6,10,9,3]. However, persistent issues such as prompt flooding, superficial questioning, and uncritical copying, limit their pedagogical impact [2,5]. Contemporary frameworks increasingly argue that *learning to interact productively with AI* is itself a critical component of digital literacy [13,11,4].

2 Best Practice

We propose **SocraticAI**, which reimagines LLMs not as answer engines but as structured tutors within a guided, metacognitively informed learning framework. The system enforces constraints that require students to articulate their reasoning before receiving feedback, reflect afterward, and operate within deliberate-use limits. Key design components include:

1. **Constrained Interaction Design:** Daily query limits (8 per student) are enforced at the system level to discourage over-reliance and encourage deliberate question formulation. Students must submit structured input consisting of their current understanding, attempted solutions, or relevant code excerpts before receiving AI feedback. The input layer includes validation checks for completeness and relevance.

2. **Guided Query Framework:** To prevent direct solution-seeking, multi-stage prompting enforces a structured dialogue:
 - (a) describe the current approach and specific confusion points,
 - (b) explain prior attempts, and
 - (c) identify the concept or implementation detail requiring clarification.
 Few-shot exemplars embedded in the system prompt illustrate productive Socratic dialogue patterns, reducing unproductive or overly general queries.
3. **RAG-Based Course Integration:** Course materials are preprocessed into a semantically indexed knowledge base. A retrieval-augmented generation (RAG) pipeline grounds AI responses in relevant lectures, assignments, or textbook excerpts, minimizing hallucinations and ensuring curricular alignment. Students thus receive feedback contextualized within their course ecosystem.
4. **Reflection and Escalation:** Each interaction concludes with reflection prompts designed to elicit awareness (such as “What did you learn?” or “What remains unclear?”). Students must summarize what they learned, identify unresolved questions, or outline next steps. These student interactions are systemically preserved for creating a record of student engagement. Additionally, When reflection fails to resolve confusion, the system supports escalation to instructors, with full conversation history preserved for context-aware intervention. This record could further help instructors identify common learning challenges and tailor follow-up guidance accordingly.
5. **System Architecture:** The system is implemented with modular, service-oriented architecture with separate services for authentication, feedback collection and handling, admin dashboard, vector retrieval, etc. This design choice facilitates scalable deployment and maintainable growth. Redis is used for real-time data storage with periodic persistence for post-analysis. The system also supports dynamic feedback tagging, and administrative monitoring via a dashboard.
6. **Implementation and Observability:** A Prometheus-based observability pipeline logs query volume, reflection quality, and escalation frequency. Technical safeguards include (i) input sanitization to prevent injection attacks, (ii) context management for long conversations, and (iii) adversarial testing of system prompts. These guardrails ensure robustness, transparency, and resilience in real-world classroom settings.

Collectively, these mechanisms transform student use of LLMs from unstructured solution-seeking into a scaffolded process that cultivates problem decomposition, metacognitive reflection, and professional-grade AI interaction skills.

3 Justification

Our approach addresses three interrelated challenges in computer science education: fostering AI literacy, sustaining cognitive engagement, and accommodating diverse learning preferences within a rigorous pedagogical framework.

1. SocraticAI develops AI literacy through structured, reflective practice with professional-grade interaction patterns. By requiring students to articulate their reasoning, clarify problem statements, and engage in guided questioning, the system models communication behaviors used in industry settings where LLMs are increasingly embedded in development pipelines. This aligns directly with ACM/IEEE curricular guidelines emphasizing responsible and transparent use of emerging technologies as part of computational ethics and software professionalism.

2. The system sustains cognitive engagement by enforcing a “think–articulate–reflect” loop. Students must explain their current understanding and attempted strategies before receiving any feedback, which reduces passive consumption and promotes active learning. Reflection prompts following each session reinforce metacognitive monitoring. In effect, SocraticAI transforms AI use from a transactional question–answer exchange into a cognitively demanding learning process.
3. The design accommodates a range of learning styles through natural language interaction while upholding consistent pedagogical standards. Novice learners benefit from conversational scaffolds that adapt to their phrasing and conceptual level, whereas advanced students use the same system to refine code efficiency or conceptual precision. By balancing flexibility with structured constraints, the system supports both equity of access and consistency of instructional rigor across diverse cohorts.

Taken together, these justifications position SocraticAI as a bridge between pedagogical integrity and technological innovation, teaching students not merely to *use* AI, but to *reason with* it.

4 Outcomes

Deployment in CS-1102 Introduction to Computer Science course at Ashoka University produced measurable and qualitative improvements in student learning behaviors. Over a three-week observation period, students exhibited a clear shift from vague, surface-level questions (e.g., “My code doesn’t work”) toward precise, decomposition-oriented inquiries such as “I implemented recursion correctly, but I’m unsure how my base case terminates.” This linguistic evolution reflects a broader cognitive shift from debugging by trial and error to analytical problem framing.

Approximately 75% of participants consistently provided reflective responses identifying specific misconceptions or next steps. Many reflections explicitly referenced conceptual insights (e.g., recognizing off-by-one errors as logic issues rather than syntax problems). Furthermore, instructors reported fewer repetitive, low-cognitive-load questions in office hours, suggesting that SocraticAI successfully offloaded routine guidance while preserving opportunities for deeper conceptual discussion.

Student feedback reinforced these findings. Survey responses indicated that the structured prompts “made me slow down and think before asking for help,” and that the reflective step “helped me understand my own gaps.” Several students described the system as “training me to ask better questions,” suggesting early evidence of growth and transferable self-regulation skills.

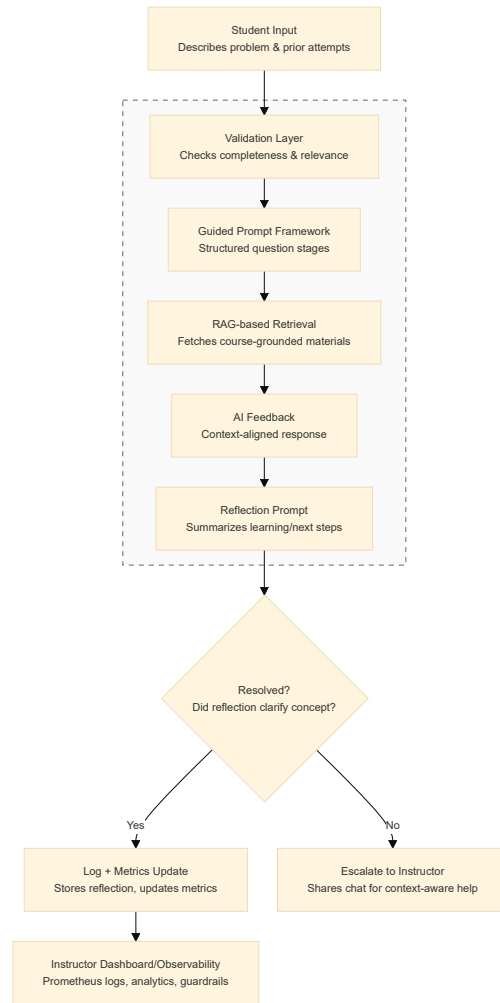


Fig. 1. System Workflow Diagram

Controlled prompt injection experiments during the summer revealed minor vulnerabilities in the retrieval layer and context management module. These findings informed ongoing improvements in input sanitation, conversation-state handling, and adversarial testing protocols. The results underscore the importance of continuous technical iteration alongside pedagogical evaluation.

5 Insights

SocraticAI enhances learning quality while reducing instructor load. By requiring students to structure and reflect on their queries, the system acts as a pre-filter for instructor intervention, ensuring that escalated cases already include context, attempted reasoning, and relevant code snippets.

Nevertheless, challenges remain. A subset of students attempted to circumvent reflective prompts or reformulate prohibited solution requests, indicating a need for adaptive constraint mechanisms and fine-tuned policy enforcement. Maintaining robust semantic grounding across diverse course materials also requires ongoing curation of the retrieval index.

6 Conclusion

Our findings demonstrate that structured constraints can fundamentally reframe the role of LLMs in computing education. Rather than banning or uncritically adopting generative AI, SocraticAI establishes a middle ground: a principled, scaffolded environment where students learn how to interact productively, ethically, and reflectively with AI systems.

The observed behavioral shifts indicate the potential of scaffolded AI tutors to foster not just competence, but cognitive maturity. Reflection prompts proved particularly effective in helping students internalize problem-solving techniques and in cultivating an awareness of their own learning processes. Looking forward, we envision expanding SocraticAI across more curricular contexts.

7 Suggestions for Others

For instructors considering similar implementations, several opportunities and pitfalls merit attention. On the opportunity side, scaffolded LLMs can scale individualized tutoring support without overwhelming teaching staff, and the required reflections generate rich data for both formative assessment and pedagogical research. Additionally, role-based dashboards provide instructors with visibility into student engagement patterns, enabling targeted interventions.

Potential pitfalls include underestimating the technical complexity of guardrails, particularly in defending against prompt injection, ensuring robust semantic retrieval, and maintaining privacy-compliant logging. Implementation also carries operational costs that must be balanced against institutional resources. To maximize effectiveness, we recommend iterative stress-testing of constraints, explicit onboarding for students to model productive interactions, and continuous monitoring of usage data to adjust limits and prompts. With these considerations, scaffolded systems like SocraticAI can be adapted to diverse institutional contexts while sustaining pedagogical integrity.

References

1. Becker, B.A., Craig, M., Denny, P., Keuning, H., Kiesler, N., Leinonen, J., Luxton-Reilly, A., Malmi, L., Prather, J., Quille, K.: Generative ai in introductory programming. In: Generative AI in Introduc-

- tory Programming (White Paper). ACM CSEd, New York, NY, USA (2023), <https://csed.acm.org/wp-content/uploads/2023/12/Generative-AI-Nov-2023-Version.pdf>, white Paper, November 2023
2. Denny, P., Kumar, V., Giacaman, N.: Conversing with copilot: Exploring prompt engineering for solving cs1 problems using natural language. In: Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1. pp. 1136–1142 (2023), <https://dl.acm.org/doi/10.1145/3545945.3569823>
 3. Finnie-Ansley, J., Denny, P., Becker, B.A., Luxton-Reilly, A., Prather, J.: The robots are coming: Exploring the implications of openai codex on introductory programming. In: Proceedings of the 24th Australasian Computing Education Conference. pp. 10–19 (2022), <https://dl.acm.org/doi/10.1145/3511861.3511863>
 4. Kazemitabaar, M., Glassman, J., Chow, R., Ma, X., Ericson, B., Weintrop, D., Grossman, T.: Studying the effect of ai code generators on supporting novice learners in introductory programming. In: Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems. pp. 1–23 (2023), <https://dl.acm.org/doi/10.1145/3544548.3580919>
 5. Lau, S., Guo, P.J.: From “ban it till we understand it” to “resistance is futile”: How university programming instructors plan to adapt as more students use ai code generation and explanation tools such as chatgpt and github copilot. In: Proceedings of the 2023 ACM Conference on International Computing Education Research - Volume 1. pp. 81–96 (2023), <https://dl.acm.org/doi/10.1145/3568813.3600138>
 6. Liffiton, M., Sheese, B.E., Savelka, J., Denny, P.: Codehelp: Using large language models with guardrails for scalable support in programming classes. In: Proceedings of the 23rd Koli Calling International Conference on Computing Education Research. pp. 1–11 (2023), <https://dl.acm.org/doi/10.1145/3631802.3631830>
 7. Liu, R., Zenke, C., Liu, C., Holmes, A., Thornton, P., Malan, D.J.: Teaching cs50 with ai: Leveraging generative artificial intelligence in computer science education. In: Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1. pp. 750–756. ACM, New York, NY, USA (2024), <https://dl.acm.org/doi/10.1145/3626252.3630958>, DOI: 10.1145/3626252.3630958
 8. Lyu, W., Wang, Y., Chung, T.R., Sun, Y., Zhang, Y.: Evaluating the effectiveness of llms in introductory computer science education: A semester-long field study. arXiv preprint arXiv:2404.13414 (2024), <https://arxiv.org/abs/2404.13414>, preprint
 9. MacNeil, S., Tran, A., Mogil, D., Bernstein, S., Ross, E., Huang, Z.: Experiences from using code explanations generated by large language models in a web software development e-book. In: Proceedings of the 53rd ACM Technical Symposium on Computer Science Education - Volume 1. pp. 931–937 (2022), <https://dl.acm.org/doi/10.1145/3545945.3569785>
 10. Prather, J., Becker, B.A., Craig, M., Denny, P., Dzugan, D., Leinonen, J.: The robots are here: Navigating the generative ai revolution in computing education. In: Proceedings of the 2023 Working Group Reports on Innovation and Technology in Computer Science Education. pp. 108–159 (2023), <https://dl.acm.org/doi/10.1145/3623762.3633499>
 11. Ray, P.P.: Chatgpt: A comprehensive review on background, applications, key challenges, bias, ethics, limitations and future scope. Internet of Things and Cyber-Physical Systems **3**, 121–154 (2023), <https://www.sciencedirect.com/science/article/pii/S266734522300024X>
 12. Sarsa, S., Denny, P., Hellas, A., Leinonen, J.: Automatic generation of programming exercises and code explanations using large language models. In: Proceedings of the 2022 ACM Conference on International Computing Education Research - Volume 1. pp. 27–43 (2022), <https://dl.acm.org/doi/10.1145/3501385.3543957>
 13. William & Mary News: The art of asking questions: Does ai in the classroom facilitate deep learning? (August 2024), <https://news.wm.edu/2024/08/01/the-art-of-asking-questions-does-ai-in-the-classroom-facilitate-deep-learning>, news article